

## PLANTILLA BASE — EJEMPLAR Y DESCRIPCIÓN TÉCNICA

Este documento corresponde al ejemplar representativo del código fuente (ANEXO 1) y a la descripción técnica o resumen funcional (ANEXO 2) del software desarrollado por el equipo indicado. Las capturas, diagramas y estructura de carpetas complementarios se adjuntan por separado como imágenes o PDF.

### ANEXO 1: EJEMPLAR DEL SOFTWARE

Título de la obra: Sistema Inteligente de Evaluación Preliminar de Problemas Visuales mediante Visión Computacional en Multi Ópticas v1.0

Tipo de software: Aplicación web cliente-servidor (plataforma web)

Lenguaje(s) de programación: PHP, JavaScript (ECMAScript)

Entorno / Framework: Laravel 12, React 18+, Vite; Laravel Sanctum; SQL (motor relacional)

Autor(es): Andy Brayan Torres Crisóstomo; Xiomara Andrea Mejía Cosios; Jasmi Janeli Álvarez Caisahuana

Empresa o titular: Multiopticas (RUC N.º 10465074511 — titular de derechos patrimoniales)

A continuación se incluye una muestra representativa del código fuente del proyecto (backend y frontend). Se han seleccionado rutas API, autenticación, gestión de pacientes, integración con servicio de IA (Gemini), persistencia en base de datos, cliente HTTP en React y generación de PDF. No se incluyen credenciales, claves de API ni datos personales reales.

#### Archivo: backend/routes/api.php

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\API\AuthController;
use App\Http\Controllers\API\PacienteController;
use App\Http\Controllers\API\EvaluacionController;
use App\Http\Controllers\API\HistorialController;
use App\Http\Controllers\API\UsuarioController;
use App\Http\Controllers\API\RetinaController;

Route::options('/retina/analizar', function() {
```

```

        return response('', 200)
        ->header('Access-Control-Allow-Origin', '*')
        ->header('Access-Control-Allow-Methods', 'POST, OPTIONS')
        ->header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
    });

Route::post('/login', [AuthController::class, 'login']);
Route::post('/usuarios/registro', [UsuarioController::class, 'registro']);

Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::get('/me', [AuthController::class, 'me']);

    Route::get('/pacientes', [PacienteController::class, 'index']);
    Route::post('/pacientes', [PacienteController::class, 'store']);
    Route::get('/pacientes/{id}', [PacienteController::class, 'show']);
    Route::put('/pacientes/{id}', [PacienteController::class, 'update']);
    Route::delete('/pacientes/{id}', [PacienteController::class, 'destroy']);

    Route::post('/evaluaciones', [EvaluacionController::class, 'store']);
    Route::get('/evaluaciones/{id}', [EvaluacionController::class, 'show']);
    Route::put('/evaluaciones/{id}/completar', [EvaluacionController::class, 'completar']);

    Route::get('/historial/paciente/{paciente_id}/pdf', [HistorialController::class, 'descargarPdfPaciente']);
    Route::get('/historial/{paciente_id}', [HistorialController::class, 'porPaciente']);
    Route::get('/historial/{historial_id}/imagen', [HistorialController::class, 'obtenerImagen']);
    Route::get('/historial/{historial_id}/pdf', [HistorialController::class, 'descargarPdf']);
    Route::delete('/historial/{historial_id}', [HistorialController::class, 'destroy']);

    Route::post('/retina/analizar', [RetinaController::class, 'analizar']);
});

```

### Archivo: backend/app/Http/Controllers/API/AuthController.php

```

<?php
namespace App\Http\Controllers\API;

use App\Http\Controllers\Controller;
use App\Models\Usuario;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;

class AuthController extends Controller
{
    public function login(Request $request)
    {
        $request->validate([
            'email' => 'required|email',
            'password' => 'required'
        ]);
    }
}

```

```

        $usuario = Usuario::where('email', $request->email)->first();

        if (!$usuario || !Hash::check($request->password, $usuario->password)) {
            return response()->json([
                'message' => 'Credenciales incorrectas'
            ], 401);
        }

        $token = $usuario->createToken('auth_token')->plainTextToken;

        return response()->json([
            'token' => $token,
            'usuario' => [
                'id' => $usuario->id,
                'nombre' => $usuario->nombre,
                'email' => $usuario->email,
                'rol' => $usuario->rol
            ]
        ]);
    }

    public function logout(Request $request)
    {
        $request->user()->currentAccessToken()->delete();
        return response()->json(['message' => 'Sesión cerrada']);
    }

    public function me(Request $request)
    {
        return response()->json($request->user());
    }
}

```

### Archivo: backend/app/Http/Controllers/API/PacienteController.php (fragmento)

```

<?php
// ... namespace e imports omitidos por brevedad ...

class PacienteController extends Controller
{
    public function index()
    {
        return response()->json(
            Paciente::orderBy('created_at', 'desc')->get()
        );
    }

    public function store(Request $request)
    {
        $request->validate([
            'nombre' => 'required|string|max:100',
            'apellido' => 'required|string|max:100',
            'dni' => 'nullable|string|max:20|unique:pacientes,dni',
            'fecha_nacimiento' => 'nullable|date',
            'genero' => 'nullable|in:M,F,otro',
            'telefono' => 'nullable|string|max:20',
            'email' => 'nullable|email|max:150|unique:pacientes,email',

```

```

        'direccion'          => 'nullable|string',
    ]]);

    $paciente = Paciente::create([
        'nombre'              => $request->nombre,
        'apellido'            => $request->apellido,
        'dni'                  => $request->dni,
        'fecha_nacimiento'    => $request->fecha_nacimiento,
        'genero'               => $request->genero,
        'telefono'             => $request->telefono,
        'email'                => $request->email,
        'direccion'            => $request->direccion,
        'registrado_por'      => $request->user()->id,
    ]);

    return response()->json($paciente, 201);
}

public function show($id)
{
    $paciente = Paciente::with([
        'evaluaciones.imagenesRetina',
        'evaluaciones.historialRetina',
        'imagenesRetina',
        'historialRetina',
    ])->findOrFail($id);

    return response()->json($paciente);
}
}

```

### Archivo: backend/app/Models/Paciente.php

```

<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Paciente extends Model
{
    protected $table = 'pacientes';

    protected $fillable = [
        'nombre', 'apellido', 'dni', 'fecha_nacimiento', 'genero',
        'telefono', 'email', 'direccion', 'registrado_por'
    ];

    public function evaluaciones()
    {
        return $this->hasMany(Evaluacion::class, 'paciente_id');
    }

    public function imagenesRetina()
    {
        return $this->hasMany(ImagenRetina::class, 'paciente_id');
    }

    public function historialRetina()
    {
    }
}

```

```

    {
        return $this->hasMany(HistorialRetina::class, 'paciente_id');
    }
}

```

### Archivo: backend/app/Http/Controllers/API/RetinaController.php (extracción representativa)

```

<?php
// Validación de entrada, codificación en base64 y prompt estructurado (ICDR / JSON)
// hacia API Gemini; fragmento interno de prompt omitido por extensión – figura
íntegro en el repositorio.

public function analizar(Request $request)
{
    $request->validate([
        'imagen' => 'required|image|mimes:jpg,jpeg,png|max:5120',
        'paciente_id' => 'required|integer|exists:pacientes,id',
    ]);

    $archivo = $request->file('imagen');
    $base64 = base64_encode(file_get_contents($archivo->getRealPath()));
    $mimeType = $archivo->getMimeType();

    $prompt = <<<'PROMPT'
[Contenido extenso: instrucciones clínicas ICDR, reglas de respuesta y esquema JSON
únicos del proyecto – ver archivo RetinaController.php en el repositorio.]
PROMPT;

    $apiKey = config('services.gemini.key');

    $response = Http::timeout(60)->post(
        "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-
flash:generateContent?key={$apiKey}",
        [
            'contents' => [[
                'parts' => [
                    [ 'inline_data' => [ 'mime_type' => $mimeType, 'data' => $base64
] ],
                    [ 'text' => $prompt ]
                ]
            ]],
            'generationConfig' => [
                'temperature' => 0.1,
                'maxOutputTokens' => 3000,
            ]
        ]
    );

    // Parseo de JSON devuelto, manejo de errores y transacción DB:
    // Evaluacion::create, ImagenRetina::create, HistorialRetina::create;
    // almacenamiento en storage/app/public/retinas; respuesta JSON al cliente.
}

```

### Archivo: backend/app/Http/Controllers/API/HistorialController.php (fragmento PDF)

```

<?php

```

```

use Barryvdh\DomPDF\Facade\Pdf;

public function descargarPdf($historial_id)
{
    $registro = HistorialRetina::with(['paciente', 'evaluacion.usuario', 'imagen'])
        ->findOrFail($historial_id);

    $imagenUrl = null;
    $ruta = $registro->imagen?->ruta_imagen;
    if ($ruta && Storage::disk('public')->exists($ruta)) {
        $imagenUrl = Storage::disk('public')->path($ruta);
    }

    $pdf = Pdf::loadView('pdf.resultado_retina', [
        'registro' => $registro,
        'imagenUrl' => $imagenUrl,
        'generadoEn' => now(),
    ])->setPaper('a4');

    $paciente = $registro->paciente;
    $nombreArchivo = 'resultado-retina-' . ($paciente?->dni ?: $registro-
->paciente_id) . '-' . $registro->id . '.pdf';

    return $pdf->download($nombreArchivo);
}

```

### Archivo: frontend/src/api/axios.js

```

import axios from 'axios'

const api = axios.create({
    baseURL: 'http://127.0.0.1:8000/api/',
    headers: { 'Content-Type': 'application/json' }
})

api.interceptors.request.use(config => {
    const token = localStorage.getItem('token')
    if (token) config.headers.Authorization = `Bearer ${token}`
    return config
})

export default api

```

### Archivo: frontend/src/components/AnalisisRetina.jsx (fragmento)

```

import { useState, useRef } from 'react'
import api from '../api/axios'

export default function AnalisisRetina({ pacienteId, nombrePaciente }) {
    const [imagen, setImagen] = useState(null)
    const [cargando, setCargando] = useState(false)
    const [reporte, setReporte] = useState(null)

    const handleAnalizar = async () => {
        if (!imagen) return
        setCargando(true)
        try {

```

```
const formData = new FormData()
formData.append('imagen', imagen)
formData.append('paciente_id', pacienteId)

const res = await api.post('retina/analizar', formData, {
  headers: { 'Content-Type': 'multipart/form-data' }
})
setReporte(res.data.reporte)
} finally {
  setCargando(false)
}
}

// ... presentación JSX: preview, tarjetas por clasificación ICDR, signos_rd, etc.
}
```

## **ANEXO 2: DESCRIPCIÓN TÉCNICA O RESUMEN FUNCIONAL**

Título del software: Sistema Inteligente de Evaluación Preliminar de Problemas Visuales mediante Visión Computacional en Multi Ópticas v1.0

Versión: 1.0

Autores: Andy Brayan Torres Crisóstomo; Xiomara Andrea Mejía Cosios; Jasmi Janeli Álvarez Caisahuana

Año de creación: 2025 – 2026

Lenguajes / tecnologías empleadas: PHP 8.x, Laravel 12, JavaScript, React, Vite, SQL, Laravel Sanctum, HTTP Client, API Google Gemini (IA generativa multimodal), DomPDF, Axios.

Tipo de obra: Plataforma web (obra de software)

### **1. Descripción general**

El software Sistema Inteligente de Evaluación Preliminar de Problemas Visuales mediante Visión Computacional en Multi Ópticas v1.0 es una aplicación web orientada al ámbito de Multi Ópticas. Permite registrar y consultar pacientes, asociar imágenes de retina, solicitar un análisis asistido mediante modelo de inteligencia artificial (visión computacional / lenguaje multimodal) con clasificación alineada a criterios de retinopatía diabética (escala tipo ICDR), conservar historial de evaluaciones y generar informes en formato PDF para respaldo documental. El sistema complementa la evaluación clínica profesional; no sustituye el juicio del especialista.

### **2. Estructura y funcionamiento**

Capa de presentación: SPA en React, empaquetada con Vite; formularios, rutas protegidas y consumo de API REST.

Capa de aplicación: backend Laravel expone controladores REST; autenticación mediante tokens Laravel Sanctum; validación de datos; orquestación del flujo de evaluación y del llamado al servicio externo de IA.



Capa de datos: base de datos relacional (SQL) con modelos Eloquent para pacientes, evaluaciones, imágenes de retina e historial; almacenamiento de archivos en disco público del framework.

Integración IA: el servidor envía la imagen y un prompt técnico estructurado al API Gemini; la respuesta es normalizada como JSON, persistida y mostrada en el cliente.

### **3. Características técnicas**

- Autenticación basada en token (Sanctum) e interceptor Axios en el frontend.
- CRUD de pacientes y relación con evaluaciones e historial de retina.
- Endpoint dedicado al análisis de retinografía con multipart/form-data.
- Clasificación, signos, urgencia y recomendación textual según respuesta del modelo.
- Descarga de PDF de resultado (DomPDF y vista Blade dedicada).
- Manejo de errores y logging en integración con IA.

### **4. Originalidad**

La originalidad reside en la implementación integrada: flujo clínico-operativo específico para Multi Ópticas, la composición de reglas y formato de intercambio con el modelo de IA, la persistencia coherente en tres entidades relacionadas (evaluación, imagen, historial), la interfaz de visualización de hallazgos y la generación automatizada de reportes en PDF acoplados al historial del paciente.

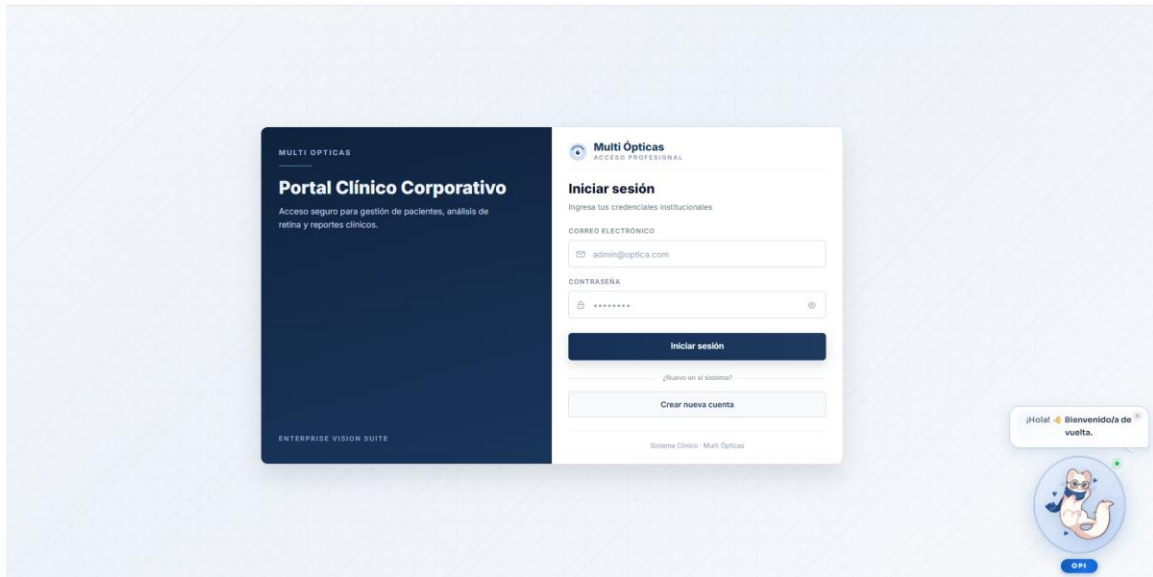
### **5. Entorno de ejecución**

- Servidor: compatible con entornos donde PHP y Composer ejecutan Laravel (Windows / Linux en desarrollo o servidor).
- Cliente: navegador moderno; build estático del frontend servido por Vite en desarrollo o por servidor web en producción.
- Base de datos: motor relacional compatible con Laravel (p. ej. MySQL/MariaDB o similar según despliegue).
- Requisitos: PHP, extensiones habituales de Laravel, Node.js para build del frontend, clave de API para Gemini en variables de entorno.

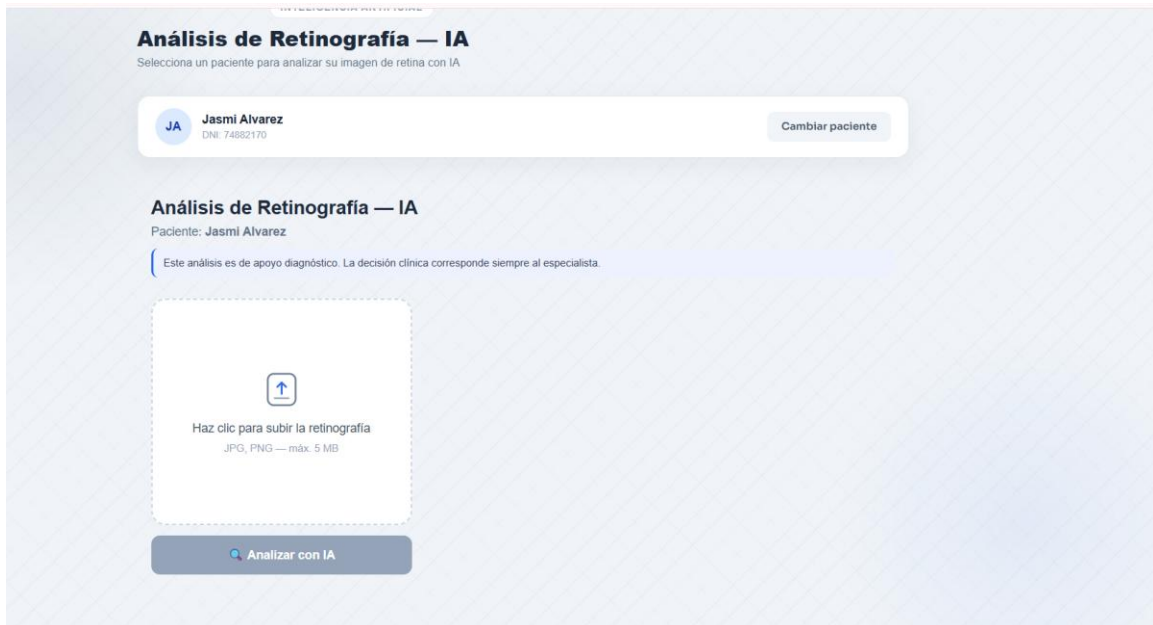
## 6. Archivos adjuntos (opcional)

Se adjuntan o incorporan por separado: capturas de login, módulo de pacientes, análisis de retina y reporte PDF; diagrama de arquitectura cliente-servidor; diagrama de flujo del análisis; estructura de carpetas backend/ y frontend/ del repositorio.

LOGIN:



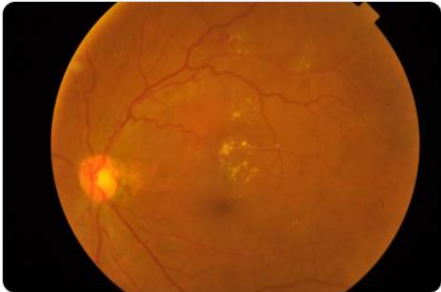
ANÁLISIS DE RETINA:



**Retinopatía Moderada**

Confianza: Alto • Urgencia: **Prioridad**

IMAGEN GUARDADA



RECOMENDACIÓN

Se recomienda una evaluación oftalmológica completa, incluyendo tomografía de coherencia óptica (OCT) para descartar edema macular diabético, y optimización del control glucémico y de otros factores de riesgo cardiovascular. Seguimiento estrecho.

SIGNOS DETECTADOS

● Microaneurismas

● Exudados duros

● Neovascularización

● Hemorragias retinianas

● Exudados blandos (algodón)

● Edema macular

HALLAZGOS OBSERVADOS

- Múltiples exudados duros, predominantemente en el polo posterior y región macular, formando agrupaciones.
- Presencia de microaneurismas dispersos, algunos asociados a los exudados.
- Vasculatura retiniana con tortuosidad leve.

REPORTE PDF:

Multi  
ÓPTICAS

Dashboard

Pacientes

A

Admin  
OPTOMETRISTA

Cerrar sesión

Volver a Reportes PDF

**Reportes PDF — Andy Torres**

DNI: 12345678 - 5 análisis

Descargar análisis seleccionado (PDF)

Descargar historial completo (PDF)

ANÁLISIS REGISTRADOS

07 may, 2026, 01:34 a. m.

**Moderada**

Confianza: Alto

Prioridad

01 may, 2026, 01:21 p. m.

**Moderada**

Confianza: Alto

Prioridad

01 may, 2026, 01:18 p. m.

**Leve**

Confianza: Medio

Rutina

01 may, 2026, 01:16 p. m.

**Moderada**

Confianza: Alto

Prioridad

27 abr, 2026, 01:07 a. m.

**Severa/Proliferativa**

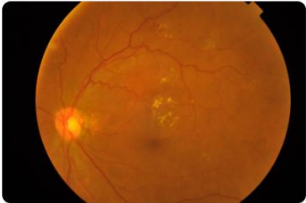
Confianza: Alto

Urgente

**Retinopatía Moderada**

Confianza: Alto • Urgencia: **Prioridad**

IMAGEN GUARDADA



RECOMENDACIÓN

Se recomienda una evaluación oftalmológica completa, incluyendo tomografía de coherencia óptica (OCT) para descartar edema macular diabético, y optimización del control glucémico y de otros factores de riesgo cardiovascular. Seguimiento estrecho.

SIGNOS DETECTADOS

● Microaneurismas

● Exudados duros

● Neovascularización

● Hemorragias retinianas

● Exudados blandos (algodón)

● Edema macular

HALLAZGOS OBSERVADOS

- Múltiples exudados duros, predominantemente en el polo posterior y región macular, formando agrupaciones.
- Presencia de microaneurismas dispersos, algunos asociados a los exudados.
- Vasculatura retiniana con tortuosidad leve.